

\$DNG Tokenomics — Dungeon Knights on Robinhood Chain

Canonical tokenomics v1.0 for mainnet deployment. This file supersedes the scattered numbers in `README.md` / `config.js` / `docs/` (see §9 for reconciled conflicts).

1. Summary

Item	Value
Token	Dungeon Token, \$DNG
Standard / decimals	ERC-20, 18
Chain	Robinhood Chain mainnet (EVM L2, Arbitrum Orbit), chain ID 4663
RPC / explorer / bridge	<code>https://rpc.mainnet.chain.robinhood.com</code> / <code>https://robinhoodchain.blockscout.com</code> / <code>https://bridge.robinhood.com/</code>
Total supply	1,000,000 \$DNG , fixed, minted once at deployment (no inflation, no burn on transfer)
Mint (summon knight)	**500 <i>DNG</i> * <i>perknight, randomrarity, paidin</i> DNG via <code>approve</code> + <code>transferFrom</code>
Reward source	On-chain Reward Pool (35% of supply); mint fees recirculate 100% into the pool
NFTs	<code>DungeonKnights</code> ERC-721 (<code>KNIGHT</code>), 6 rarities, stats generated on-chain
Testnet (live now)	Robinhood testnet chain ID 46630 , NFT <code>0xEA37B1D036a880DfF372bCdd8b2A3AEeE01e55A</code> , 0.001 ETH/mint — gameplay only, no \$DNG

2. Design goals

1. **Fair ROI**: an Uncommon knight breaks even in **25 dungeon clears**; every tier ROIs (worst case Common: 42).
2. **Sustainable loop**: mints fund rewards — no perpetual emission; the pool cannot pay out what mints haven't put in (plus seed).
3. **Simple v1**: no staking, no marketplace cut, no tax on transfer. Sinks are gameplay (mints, future upgrades).
4. **Robinhood-native**: low L2 gas, ETH as gas token, Blockscout verification, plain MetaMask/RainbowKit UX.

3. Supply & distribution (1,000,000 \$DNG)

Bucket	%	Amount	Custody / lock	Purpose
Reward Pool	35%	350,000	<code>RewardDistributor</code> contract (or team multisig pre-audit)	Pays dungeon-clear claims
Liquidity	30%	300,000	DEX pool + LP lock 12 mo	Tradability \$DNG/ETH
Treasury	15%	150,000	Multisig, quarterly vest	Dev, audits, servers, art
Marketing / community	10%	100,000	Multisig + streaming	Tournaments, creators, quests
Team	10%	100,000	12-mo linear vest, 6-mo cliff	Founders/devs

Rules: fixed supply; ownership renounced or timelocked after distribution; LP lock proof published; team/marketing wallets public in §8.

4. Rarity, drops & rewards (canonical)

On-chain roll (`_rollRarity` , d1000) and frontend `characters.js` agree — this table is law:

Rarity	Drop	Mint odds	Reward / dungeon clear	Clears to ROI (500 cost)
Common	65%	650/1000	12 \$DNG	42
Uncommon	30%→**20%** ¹	200/1000	20 \$DNG	25  headline
Rare	15%→**10%** ¹	100/1000	36 \$DNG	14
Epic	4%→**4.5%** ¹	45/1000	60 \$DNG	9
Legendary	1%→**1.2%** ¹	12/1000	100 \$DNG	5
Mythic	—→**0.3%** ¹	3/1000	150 \$DNG	4

¹ README predates Mythic and quotes 50/30/15/4/1. **Code + contract (65/20/10/4.5/1.2/0.3) wins**; README to be updated.

Expected reward per fresh mint $\approx 0.65 \cdot 12 + 0.20 \cdot 20 + 0.10 \cdot 36 + 0.045 \cdot 60 + 0.012 \cdot 100 + 0.003 \cdot 150 \approx$ ****19.75 \$DNG/clear** * ***.Mintcost500DNG** $\Rightarrow$ expected payback $\approx$ 25.3 clears — matches the "25 dungeons" promise.

Knight stat multipliers (on-chain `_getRarityMultiplier` / frontend): 1.0 / 1.5 / 2.2 / 3.5 / 5.0 / 8.0 — gameplay speed only, rewards are table-fixed.

5. Mint & reward flows (mainnet)

Mint (ERC-20 path, replaces testnet ETH path):

1. Player `approve(NFT_CONTRACT, 500 DNG)` .
2. `mintKnight()` pulls `transferFrom(player, RewardPool, 500 DNG)` then mints ERC-721 with rarity + stats, emits `KnightMinted` .
3. Batch `mintKnights(n)` for $n = 1\text{--}10$ (front page caps qty at 5; contract allows 10).

Earn (v1 — claim model):

- Off-chain game tracks clears (see `docs/IDLE_MECHANICS.md` : ~ 10 gold/clear off-chain $\approx$ table rewards on-chain; Common solo $\approx$ 5h/clear, Mythic $\approx$ 25min).
- Player submits clear claim; `RewardDistributor` verifies (v1: backend-signer signature; v2: ZK/oracle) and pays table amount for the knight's rarity from the pool.
- Anti-farm: one claim per knight per dungeon-cycle id, cooldown = expected clear time, cap 15 deployed knights (matches `game.js`).

Pool math: each mint injects 500; each clear emits 12–150. Pool grows while mint velocity > reward velocity. With seed 350k, the pool survives even a mint drought for $\approx 17,700$ Uncommon-equivalent clears. If pool < threshold, rewards queue (no minting of new \$DNG — hardness guarantee).

6. Contracts to deploy (Robinhood mainnet)

1. **DungeonToken (\$DNG)** — OpenZeppelin ERC-20, fixed 1M, 18 decimals.
2. **DungeonKnights (ERC-721)** — as in `docs/DEPLOYMENT_GUIDE.md` **plus** the `docs/MAINNET-DEPLOYMENT.md` ERC-20 edit: `IERC20 dungeonToken` in constructor, `mintKnight()` non-payable via `transferFrom` , `mintPrice = 500 ether` , rarity/stat logic unchanged.

3. **RewardDistributor** (new, v1 minimal): holds 350k, `claim(dungeonId, knightId, sig)`, owner = multisig, pause + per-knight cooldown.
4. Optional at launch: `VestingWallet` (team), LP locker proof.

Hardhat network (already documented): `robinhoodMainnet { url: rpc.mainnet..., chainId: 4663, gasPrice: 1 gwei }`; verify via Blockscout custom chain (`robinhoodchain.blockscout.com/api`).

7. Frontend wiring (`js/web3/config.js` changes)

```
USE_MAINNET: true,
TOKEN: { ... , mainnetAddress: '0x<DEPLOYED_DNG>' },
NFT_CONTRACTS: { testnet: '0xEA37...e55A', mainnet: '0x<DEPLOYED_NFT>' },
MINT_PRICE: { testnet: '0.001', mainnet: '500' }, // ← change '100' → '500'
```

Plus add `REWARD_DISTRIBUTOR: { mainnet: '0x<...>' }` and the claim call in `mint.js` / `game.js` completion modal (`nextDungeon` / `handleDungeonCleared` are the hook points).

8. Deployment checklist (mainnet)

- ☐ Deploy \$DNG, mint 1M to deployer, distribute per \$3 (tx hashes recorded below)
- ☐ Deploy NFT with \$DNG address in constructor; `setMintPrice(500 ether)`
- ☐ Deploy RewardDistributor, fund 350,000 \$DNG; seed liquidity 300k + lock LP
- ☐ Verify all three on Blockscout; publish ABI + addresses here:

Contract	Address	Tx
\$DNG ERC-20	0x...	...
DungeonKnights ERC-721	0x...	...
RewardDistributor	0x...	...
LP lock proof	—	...

- ☐ Update `config.js` (§7), `USE_MAINNET: true`, `vercel --prod`, test: connect → approve → mint → clear → claim on mainnet with a throwaway wallet first
- ☐ Renounce / timelock token ownership; multisig treasury

9. Reconciled conflicts (so future-you isn't confused)

- Mint price: `config.js` said **100**, README **500** → **500 wins** (\$4 ROI math only works at 500).
- Rarity: README 5-tier vs code/contract 6-tier → **6-tier wins**; Mythic reward set to 150 (4 clears to ROI).
- Squad size: README 5 vs `game.js` 15 → gameplay allows 15; rewards are per-knight so economy is unaffected, but claim caps should assume 15.
- Off-chain "gold" (~10/dungeon) vs on-chain $DNG(12-150)$: *gold is the testnet play — money*; DNG table governs mainnet claims.

10. Risks & mitigations

- Reward-pool drain if power users farm with Mythic squads → cooldowns + per-cycle claim ids + pool-low queue; tune table by multisig within ±20% bounds published here.
- \$DNG volatility breaks ROI promise → ROI quoted in **clears**, not USD; revisit table quarterly via governance note.
- Oracle trust (v1 backend signer) → publish signer, rotate keys, move to on-chain dungeon commitments in v2.
- Contract risk → audit before seeding full 350k (stage funding: 50k → 350k); bug-pause in distributor.